



Formation Kubernetes

Session 7 — Introduction à Terraform et IaC

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session



Le problème : UI vs Code

Pourquoi les clics ne passent pas à l'échelle

10 min



Outils IaC

Terraform, OpenTofu, Pulumi — comparaison et choix

10 min



Concepts Terraform

Resource, Provider, Variable, Output, Module

15 min



Le cycle Terraform

init → plan → apply → destroy — démo pas à pas

10 min



State : le nerf de la guerre

tfstate, local vs remote, locking, pièges

15 min

Recap — Session 6



Secret K8s

Stockage en etcd, monté en volume ou env var



ExternalSecret

Sync depuis Vault, Azure Key Vault, GCP Secret Manager



Vault intro

Gestion centralisée des secrets, audit trail, rotation



CSI vs ESO

CSI = montage direct (recommandé), ESO = sync K8s



Problème clé

Les secrets sont en clair dans etcd — comment les protéger ?

Question : Comment reproduire une infra entière sans cliquer dans la console ?

Le problème : UI vs Code

Approche UI

Console GCP, clicks dans le UI

- Pas reproductible
- Pas versionné (pas de git)
- Config drift = enfer
- Pas de review process
- Scalabilité zéro
- Audit trail faible

Approche Code (IaC)

Terraform / HCL dans git

- Reproductible et idempotent
- Versionné avec git — history complet
- Plan avant d'appliquer
- Review process : PR + approvals
- Scalable : clone + adapt + apply
- Audit : qui a changé quoi et quand

"Tout ce qui n'est pas versionné en git est un accident en attente d'arriver"

Les outils IaC

Terraform

HCL

✓ Avantages

Multi-cloud
Large community
Mature ecosystem

✗ Inconvénients

BSL license (restrictive)
Verbose for large configs

Choix: Terraform pour ce cours

OpenTofu

HCL

✓ Avantages

Fork open source
100% compatible Terraform
Pas de license concern

✗ Inconvénients

Communauté plus petite

Alternative libre à Terraform

Pulumi

Python/TS/Go

✓ Avantages

Code natif (pas de DSL)
Réutilisabilité haute
SDK complet

✗ Inconvénients

Courbe apprentissage plus raide
Moins mature

Pour développeurs qui aiment coder

Autres outils IaC

CloudFormation (YAML/JSON)

Crossplane (K8s CRDs)

Terraform – Les concepts clés

Provider

Pluggable : GCP, AWS, Azure, Kubernetes, Helm...

```
provider "google" { project = "my-gcp-project" }
```

Resource

Objet managé : GCS bucket, VM, Service K8s, etc.

```
resource "google_storage_bucket" "data" {  
  name = "my-bucket"  
}
```

Variable

Input paramétrables, définis en .tfvars

```
variable "app_name" {  
  type = string  
  default = "myapp"  
}
```

Output

Exporte des valeurs pour les autres configs

```
output "bucket_name" {  
  value = google_storage_bucket.data.name  
}
```

Module

Réutilisable : groupe de ressources + logique

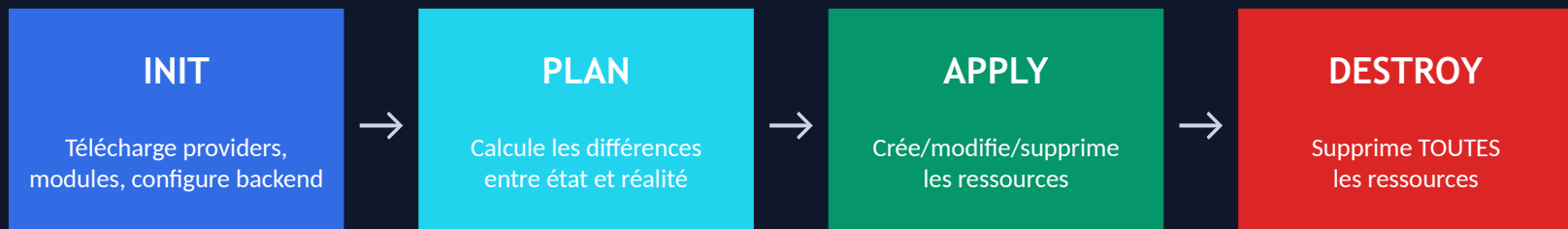
```
module "database" {  
  source = "../modules/postgresql"  
  app_name = var.app_name  
}
```

Data Source

Lit données existantes (pas créé par TF)

```
data "google_compute_zones" "available" {  
  project = var.project  
}
```

Le cycle Terraform



↪ Boucle de feedback

Workflow typique :

1. `terraform init`
2. `terraform plan -out=tfplan`
3. Review le plan (quoi va changer ?)
4. `terraform apply tfplan`
5. Itérer : edit config → plan → apply
6. `terraform destroy` (cleanup)

State : Le nerf de la guerre

Qu'est-ce que le state ?

Fichier JSON qui mappe: resource → ID cloud + attributs

Exemple:

```
resource "google_storage_bucket" "data"
```

→ ID: gs://my-bucket-prod

→ Attributs: location, versioning, etc.

Où stocker le state ?

✗ LOCAL (dev uniquement)

terraform.tfstate en disque local

Problème : pas partageable, facile de perdre

✓ REMOTE BACKEND (prod + équipe)

GCS, S3, Terraform Cloud

→ partagé + verrouillé en équipe

⚠ Le pire scénario : Race condition

1. Alice lance: terraform apply
2. Bob SIMULTANÉMENT lance: terraform apply
3. Tous les deux modifient le state → ÉTAT CORROMPU
4. Prochaine modification → catastrophe

SOLUTION: State Locking (GCS + DynamoDB / Terraform Cloud), et un SEUL CI deploy à la fois

Pièges et bonnes pratiques

Secret dans le state

❌ Mauvais:

```
password = var.db_password  
→ lisible en clair en state
```

✓ Bien:

Utiliser CSI driver ou
externe store (Vault)

Commit .tfstate en git

❌ Mauvais:

```
git add terraform.tfstate  
→ secrets exposés
```

✓ Bien:

```
.gitignore terraform.tfstate  
State toujours remote
```

Pas de plan avant apply

❌ Mauvais:

```
terraform apply  
→ surprise !
```

✓ Bien:

```
plan -out=tfplan  
apply tfplan
```

terraform import

❌ Mauvais:

Créer manuelle →
terraform ne sait pas

✓ Bien:

```
terraform import  
type.name id
```

Règles d'or : DRY (modules), plan before apply, remote state + locking, secrets in Vault

Architecture du TP – Du simple au multi-environnement

Phase 1 – Simple resource

Créer un bucket Google Cloud Storage

- main.tf : resource "google_storage_bucket"
- variables.tf : name, location
- outputs.tf : bucket_name, bucket_url
- terraform.tfvars : env-specific

Apprentissage : init, plan, apply, destroy

- Variables et interpolation
- State local
- destroy pour le cleanup
- Passer les values

Phase 2 – Module réutilisable

Refactoriser le bucket en module

- modules/gcs_bucket/main.tf
- variables.tf : paramètres
- outputs.tf : exporte les infos
- le root module appelle le module

Apprentissage : DRY, composition

- Plusieurs buckets, même module
- Output forwarding
- Module source (local vs registry)

Phase 3 – Multi-environnement

Structurer pour staging / prod

- environments/staging/
- environments/production/
- backend différent par env
- terraform.tfvars par env

Apprentissage : backend distant + équipe

- Backend remote (GCS) partagé
- State locking → collaboration
- State isolé par environnement
- Prêt pour la CI/CD

Démo live — Infra RFLKT

cloud-infrastructure/

Makefile

Commands: init, plan, apply, auth...

terraform.tf

Provider + backend config

environments/

Par environnement (staging, production)

 staging/main.tf

 ~1200 lignes : toutes les ressources staging

 production/main.tf

 Pareil pour prod

modules/

Composants réutilisables

 cluster/

 GKE cluster config

 application/

 Generic app deployment (K8s + Ingress)

 database/

 Cloud SQL PostgreSQL

 traefik/

 Helm + Traefik config

- ✓ State distant : GCS backend + locking
- ✓ Multi-env : environ/ pour staging et prod — pas de copier-coller
- ✓ Modules réutilisables : application/ appelé N fois avec params différents

Pièges avancés et bonnes pratiques



Config drift

Quelqu'un modifie au cloud sans TF → next plan voit diff

→ `terraform refresh / plan -refresh=true`, jamais modifier console



State vieilli

Pas lancé terraform depuis 2 mois → state ≠ réalité

→ `terraform plan` régulièrement, CI/CD rend ça auto



Module source instable

Module git sans version → HEAD change, breaking changes

→ `version =` ou `git ref exact : ?ref=v1.0`



Destroy accident

`terraform destroy -auto-approve` → BAM, tout parti

→ Jamais `-auto-approve` en prod, reverser state + manual fix



Récap & Mini-défi



laC = Infrastructure écrite en code, versionné, reviewable



Terraform : providers, resources, variables, outputs, modules



Cycle : init → plan → apply → iterate. JAMAIS apply sans plan



State = source de vérité. Remote backend + locking indispensable en équipe

Mini-défi pour la prochaine session

1. Créez une resource GCS bucket avec Terraform en local
2. Utilisez variables.tf pour paramétrer name et location
3. Créez outputs.tf pour exporter l'URL de la bucket
4. Exécutez init → plan → apply → destroy